ChartNav — Workflow State Machine Build

Phase: strict encounter state machine + filtered listing
Repo: `thekidd2227/chartnav-platform` · Generated 2026-04-17

Contents

1. Executive summary
2. 01 — Current state
3. 02 — Workflow state machine
4. 03 — API endpoints
5. 04 — Data model
6. 05 — Build log
7. 06 — Known gaps
8. Diagram — System architecture
9. Diagram — Encounter status machine
10. Diagram — ER
11. Diagram — API / data flow

Executive summary

This document consolidates the ChartNav *workflow state machine + filtering*

phase. It covers everything delivered in this phase:

- **Strict state machine** on `POST /encounters/{id}/status` — only the documented forward and rework edges succeed; everything else returns `400 invalid_transition` with an explicit message.
- **Filtered listing** on `GET /encounters?organization_id=&location_id=&status=&provider_name=` — all filters combinable, parameterized SQL, deterministic order.
- **Expanded seed** — two idempotent encounters covering `in_progress` and `review_needed`, each with a realistic event history.
- **Living documentation** — Mermaid diagrams for architecture, ER, status machine, and sequence flow; plus per-concern markdown docs.
- **Verification evidence** — every filter and every state transition case was exercised locally against the running API; matrices appear in §06.

Preserved untouched: `/health`, `/`, `/organizations`, `/locations`, `/users`,

the GET/POST event endpoints, Alembic history, and the SQLite dev workflow.

01 — Current state

ChartNav — Current State

As of: 2026-04-17 (phase: workflow state machine + filtering)

Repo layout (relevant)

```
chartnav-platform/
├── apps/
│   ├── api/                          # FastAPI service (SQLite local)
│   │   ├── app/main.py               # app factory, CORS, router mount
│   │   ├── app/api/routes.py         # all HTTP handlers
│   │   ├── alembic.ini
│   │   ├── alembic/env.py
│   │   ├── alembic/versions/         # 2 migrations
│   │   ├── scripts_seed.py           # idempotent demo seed
│   │   └── pyproject.toml
│   └── web/                          # Vite shell (untouched this phase)
├── infra/docker/docker-compose.yml
└── docs/
    ├── build/                        # living docs (this set)
    ├── diagrams/                     # Mermaid sources
    ├── final/                        # consolidated HTML/PDF
    └── releases/
```

Runtime baseline

- Python 3.11+, FastAPI, raw `sqlite3` driver (no ORM yet).
- SQLite file resolved at `apps/api/chartnav.db` (gitignored).
- Alembic history: `43ccbf363a8f → a1b2c3d4e5f6` (head).
- CORS wide-open (`*`) for local dev.

Verified working endpoints (preserved)

- `GET /health` , `GET /`
- `GET /organizations` , `GET /locations` , `GET /users`
- `GET /encounters` (now with filters — see `03-api-endpoints.md`)

- `GET /encounters/{id}`
- `GET /encounters/{id}/events`
- `POST /encounters`
- `POST /encounters/{id}/events`
- `POST /encounters/{id}/status` (now strict state machine — see `02-workflow-state-machine.md`)

Demo data shipped by seed

id	patient_id	provider	status	events
1	PT-1001	Dr. Carter	in_progress	3
2	PT-1002	Dr. Patel	review_needed	5

See `04-data-model.md` and `05-build-log.md` for details.

02 — Workflow state machine

Encounter Workflow State Machine

Status values

Status	Meaning
`scheduled`	Visit is booked, patient has not arrived.
`in_progress`	Provider is seeing the patient / actively charting.
`draft_ready`	Note draft has been generated and is awaiting review.
`review_needed`	Reviewer flagged the note — needs revision or sign-off.
`completed`	Note signed off; encounter closed. **Terminal.**

Allowed transitions

Forward flow:

```
scheduled → in_progress → draft_ready → review_needed → completed
```

Explicit rework transitions (documented + enforced):

From	To	Purpose
`draft_ready`	`in_progress`	Draft rejected, return to charting.
`review_needed`	`draft_ready`	Reviewer kicks note back for rewrite.

Rejected transitions (examples that return HTTP 400)

- `scheduled → completed`
- `scheduled → review_needed`
- `scheduled → draft_ready`
- `in_progress → scheduled`
- `in_progress → completed` (must pass through draft_ready + review_needed)

- `completed → anything` (terminal)
- Any unknown status string (`invalid_status`)

Error body format:

```
{"detail":"invalid_transition: in_progress -> completed is not permitted; allowed next s
```

Idempotency

Posting the **same** status the encounter is already in is a no-op: the row

is returned unchanged and **no** `status_changed` event is appended.

Side effects on successful transition

Transition target	Timestamp action
`in_progress`	set `started_at = now()` if currently NULL
`completed`	set `completed_at = now()` ; also set `started_at = now()` if it was NULL
other	no timestamp mutation

Every successful transition appends a `workflow_events` row:

```
{
  "event_type": "status_changed",
  "event_data": {"old_status": "...", "new_status": "..."}
}
```

Diagram

See `docs/diagrams/encounter-status-machine.md` (Mermaid).

Source of truth

`apps/api/app/api/routes.py` :

- `ALLOWED_STATUSES` — the 5 permitted status strings.

- `ALLOWED_TRANSITIONS` — `dict[str, set[str]]`, one entry per source state.
- `update_encounter_status()` — the single enforcement point.

API Endpoints

Base URL (local dev): `http://127.0.0.1:8000` (or whatever port uvicorn binds).

System

Method	Path	Response
GET	`/health`	`{"status":"ok"}`
GET	`/`	`{"service":"chartnav-api","version":"0.1.0"}`

Org / location / user (read-only)

Method	Path	Notes
GET	`/organizations`	Ordered by id.
GET	`/locations`	Ordered by id.
GET	`/users`	Ordered by id.

Encounters

GET `/encounters`

Query parameters (all optional, combinable, AND-ed):

Param	Type	Notes
`organization_id`	int≥1	Exact match.
`location_id`	int≥1	Exact match.
`status`	string	Must be in `ALLOWED_STATUSES` or 400.
`provider_name`	string	Exact match (case-sensitive).

Ordering: `ORDER BY id`. All SQL uses parameterized `?` placeholders.

GET `/encounters/{encounter_id}`

`404 encounter_not_found` if missing.

GET `/encounters/{encounter_id}/events`

`404 encounter_not_found` if the parent encounter does not exist.

Returns events oldest-first (`ORDER BY id`). `event_data` is JSON-parsed

before being returned to the client when it is valid JSON.

POST `/encounters`

Body:

```
{
  "organization_id": 1,
  "location_id": 1,
  "patient_identifier": "PT-1003",
  "patient_name": "Alex Chen",
  "provider_name": "Dr. Ortiz",
  "scheduled_at": "2026-04-18T14:30:00Z",
  "status": "scheduled"
}
```

Rules:

- `status` defaults to `"scheduled"` and may only be `scheduled` or `in_progress` at creation (deeper states can't be forged).
- `organization_id` + `location_id` must exist and the location must belong to the organization → otherwise 400.
- A `workflow_events` row `encounter_created` is appended automatically.
- Returns the full encounter row (201).

POST `/encounters/{encounter_id}/events`

Body:

```
{ "event_type": "note_draft_completed", "event_data": { "template": "glaucoma-initial" }
```

`event_data` may be any JSON value (object/array/string/number/null). Objects

and arrays are stored as canonical JSON (sorted keys). Strings pass through

as-is.

POST `/encounters/{encounter_id}/status`

Body: `{ "status": "draft_ready" }`.

- Validated against `ALLOWED_STATUSES` and `ALLOWED_TRANSITIONS`

(`02-workflow-state-machine.md`).

- Same-status post is a no-op (no event recorded).
- On success, stamps `started_at` / `completed_at` per lifecycle rules

and appends a `status_changed` event.

Error examples:

- `400 invalid_status` — unknown string.
- `400 invalid_transition` — disallowed edge in the state machine.
- `404 encounter_not_found` — no such encounter.

Data Model

SQLite (local dev). Schema reproduced by Alembic migrations

`43ccbf363a8f → a1b2c3d4e5f6`.

Tables

`organizations`

column	type	constraints
id	INTEGER	PK
name	VARCHAR(255)	NOT NULL
slug	VARCHAR(255)	NOT NULL, UNIQUE
created_at	DATETIME	NOT NULL, default `now()`

`locations`

column	type	constraints
id	INTEGER	PK
organization_id	INTEGER	NOT NULL, FK → organizations(id)
name	VARCHAR	NOT NULL
created_at	DATETIME	NOT NULL, default `now()`

`users`

column	type	constraints
id	INTEGER	PK
organization_id	INTEGER	NOT NULL, FK → organizations(id)
email	VARCHAR	NOT NULL, UNIQUE
full_name	VARCHAR	NULL

column	type	constraints
role	VARCHAR	NOT NULL, default `"admin"`
created_at	DATETIME	NOT NULL, default `now()`

`encounters`

column	type	constraints
id	INTEGER	PK
organization_id	INTEGER	NOT NULL, FK → organizations(id), indexed
location_id	INTEGER	NOT NULL, FK → locations(id), indexed
patient_identifier	VARCHAR	NOT NULL, indexed
patient_name	VARCHAR	NULL
provider_name	VARCHAR	NOT NULL
status	VARCHAR	NOT NULL, default `"scheduled"`, indexed
scheduled_at	DATETIME	NULL
started_at	DATETIME	NULL (stamped on entering `in_progress`)
completed_at	DATETIME	NULL (stamped on entering `completed`)
created_at	DATETIME	NOT NULL, default `now()`

`workflow_events`

column	type	constraints
id	INTEGER	PK
encounter_id	INTEGER	NOT NULL, FK → encounters(id), indexed
event_type	VARCHAR	NOT NULL, indexed
event_data	TEXT	NULL (canonical JSON string)
created_at	DATETIME	NOT NULL, default `now()`

Relationships

```
organizations 1──┬──* locations
```

```
                 └─* users
                 └─* encounters —* workflow_events
locations     1—* encounters
```

See `docs/diagrams/er-diagram.md` for a Mermaid ER rendering.

Event taxonomy (emitted automatically)

event_type	When	Shape
`encounter_created`	On `POST /encounters`	`{"status": "<initial>"}`
`status_changed`	On every successful status transition	`{"old_status": "...", "new_status": "..."}`
user-supplied	`POST /encounters/{id}/events`	Any JSON

Seeded demo data

Two encounters are seeded for `demo-eye-clinic` / `Main Clinic`:

id	patient	provider	status	event count	lifecycle covered
1	PT-1001	Dr. Carter	`in_progress`	3	scheduled → in_progress
2	PT-1002	Dr. Patel	`review_needed`	5	scheduled → in_progress → draft_ready → review_needed

05 — Build log

Build Log

Reverse-chronological. Each entry is a concrete, verifiable step.

2026-04-17 — Phase: workflow state machine + filtering

Step 1 — Baseline inspection

- Confirmed current migrations, seed, and routes compile and run as documented in `01-current-state.md`.
- Confirmed Alembic head: `a1b2c3d4e5f6`.
- No ORM in use — raw `sqlite3` calls via helpers in `routes.py`.

Step 2 — Strict status state machine

- Added `ALLOWED_TRANSITIONS` map in `apps/api/app/api/routes.py`.
- `update_encounter_status()` now:
- Validates `status` is in `ALLOWED_STATUSES` → else 400.
- Treats same-status post as a no-op (no event written).
- Validates the source→target edge is in `ALLOWED_TRANSITIONS` → else 400 with explicit message.
- Only stamps `started_at` on entry to `in_progress` if currently NULL.
- Stamps `completed_at` on entry to `completed` (and backfills `started_at` if null).
- Appends `status_changed` event with `old_status` / `new_status` (replacing the prior `from` / `to` keys).
- `POST /encounters` now restricts initial status to `scheduled` or `in_progress` so deeper states can't be forged at creation.

Step 3 — Encounter filtering

- `GET /encounters` accepts `organization_id`, `location_id`, `status`, `provider_name`.
- All filters use parameterized SQL; clauses AND-ed; ordering preserved.
- Invalid `status` filter → 400 with enumerated allowed values.

Step 4 — Expanded seed

- `scripts_seed.py` now seeds **two** encounters:
- PT-1001 at `in_progress` with 3 events (create, status_changed, note_draft_requested).
- PT-1002 at `review_needed` with 5 events walking the full forward flow.
- Seed remains idempotent; both encounters and their events use composite uniqueness checks before insert.
- Timestamp handling at seed time: `started_at = CURRENT_TIMESTAMP` for any status that implies the visit began; `completed_at` only for `completed`.

Step 5 — Local verification

- `rm -f chartnav.db && alembic upgrade head` → both migrations ran cleanly.
- `python scripts_seed.py` ran twice; event counts stable at 3 / 5.
- `uvicorn` boot clean; all baseline endpoints returned 200.
- Filter matrix verified — see `06-known-gaps.md` for matrix.
- State-machine matrix verified — 4 valid transitions 200, 4 invalid transitions 400 with explicit detail.
- OpenAPI now exposes the four new query parameters on `GET /encounters`.

Step 6 — Documentation

- Created `docs/build/*` living docs.
- Created `docs/diagrams/*` Mermaid sources (architecture, ER, status machine, sequence).
- Generated `docs/final/chartnav-workflow-state-machine-build.html` and `.pdf`.

Step 7 — Git hygiene

- `chartnav.db` is gitignored; not committed.
- `.venv/` is gitignored.
- Committed source + docs in a single atomic commit.

Known Gaps

Verification matrix (proof this phase shipped)

Filter matrix — `GET /encounters?...`

Query	Expected ids	Actual
`status=in_progress`	`[1]`	`[1]`
`status=review_needed`	`[2]`	`[2]`
`provider_name=Dr. Patel`	`[2]`	`[2]`
`organization_id=1&location_id=1&status=in_progress`	`[1]`	`[1]`
`organization_id=999`	`[]`	`[]`
`status=bogus`	400	400

State machine matrix — `POST /encounters/{id}/status`

Encounter	From	To	Expected	Actual
1	in_progress	draft_ready	200	200
1	draft_ready	in_progress	200	200
1	in_progress	completed	400	400
1	in_progress	scheduled	400	400
2	review_needed	completed	200	200
2	completed	in_progress	400	400
1	in_progress	`"whatever"`	400	400

Real gaps (prioritized)

1. **No ORM / no sessions** — routes still use raw `sqlite3` with per-request connections. Works for single-file SQLite; will not scale cleanly to Postgres or multi-writer.

2. **No auth / no tenant scoping** — any caller sees any org's data. Needs `organization_id` derived from authenticated identity, not client input.

3. **No pagination** — `GET /encounters` returns all matching rows. Needs `limit`, `cursor`, and a stable order.

4. **No update / delete paths** — encounters cannot be edited (patient name typo) or cancelled. Need `PATCH /encounters/{id}` and a `cancelled` status (or soft-delete).

5. **No automated tests** — verification is manual curl matrices. Need pytest + httpx `TestClient` covering state machine + filters.

6. **Free-form `event_data`** — no per-event_type schema validation.

7. **SQLite-only** — Alembic constructs are SQLite-compatible but Postgres parity hasn't been exercised (docker compose has a Postgres intent but api still points at SQLite).

8. **Frontend not wired** — `apps/web` does not yet consume `/encounters`.

9. **CORS wide-open** (`allow_origins=["*"]`) — fine for dev, must be tightened before any hosted deploy.

10. **No request IDs / structured logging** — hard to trace a specific 400 in a busy log.

System Architecture

Diagram — Encounter status machine

Encounter Status State Machine

stateDiagram-v2 [*] --> scheduled [*] --> in_progress : walk-in scheduled --> in_progress in_progress --> draft_ready draft_ready --> review_needed draft_ready --> in_progress : rework review_needed --> completed review_needed --> draft_ready : kick back completed --> [*]

Legend: any edge not shown is rejected at `POST /encounters/{id}/status`

with HTTP 400 `invalid_transition`.

ER Diagram

erDiagram organizations ||--o{ locations : "has" organizations ||--o{ users : "employs" organizations ||--o{ encounters : "owns" locations ||--o{ encounters : "hosts" encounters ||--o{ workflow_events : "emits" organizations { int id PK string name string slug UK datetime created_at } locations { int id PK int organization_id FK string name datetime created_at } users { int id PK int organization_id FK string email UK string full_name string role datetime created_at } encounters { int id PK int organization_id FK int location_id FK string patient_identifier string patient_name string provider_name string status datetime scheduled_at datetime started_at datetime completed_at datetime created_at } workflow_events { int id PK int encounter_id FK string event_type text event_data datetime created_at }

Diagram — API / data flow

API / Data Flow

sequenceDiagram autonumber participant C as Client participant A as FastAPI / routes.py participant SM as State Machine participant DB as SQLite Note over C,DB: Happy-path: advance an encounter C->>A: POST /encounters/{id}/status {"status":"draft_ready"} A->>DB: SELECT status, started_at, completed_at FROM encounters WHERE id=? DB-->>A: current row A->>SM: is (in_progress → draft_ready) allowed? SM-->>A: yes A->>DB: UPDATE encounters SET status=?, started_at=?, completed_at=? WHERE id=? A->>DB: INSERT INTO workflow_events (status_changed, {old,new}) DB-->>A: committed A-->>C: 200 updated encounter row Note over C,DB: Rejected transition C->>A: POST /encounters/{id}/status {"status":"completed"} A->>DB: SELECT status... A->>SM: is (in_progress → completed) allowed? SM-->>A: no A-->>C: 400 invalid_transition (with allowed next states) Note over C,DB: Filtered list C->>A: GET /encounters?status=in_progress&provider_name=Dr.%20Carter A->>DB: SELECT ... WHERE status=? AND provider_name=? ORDER BY id DB-->>A: rows A-->>C: 200 [rows]